Documentation du projet GEA

Contexte du projet

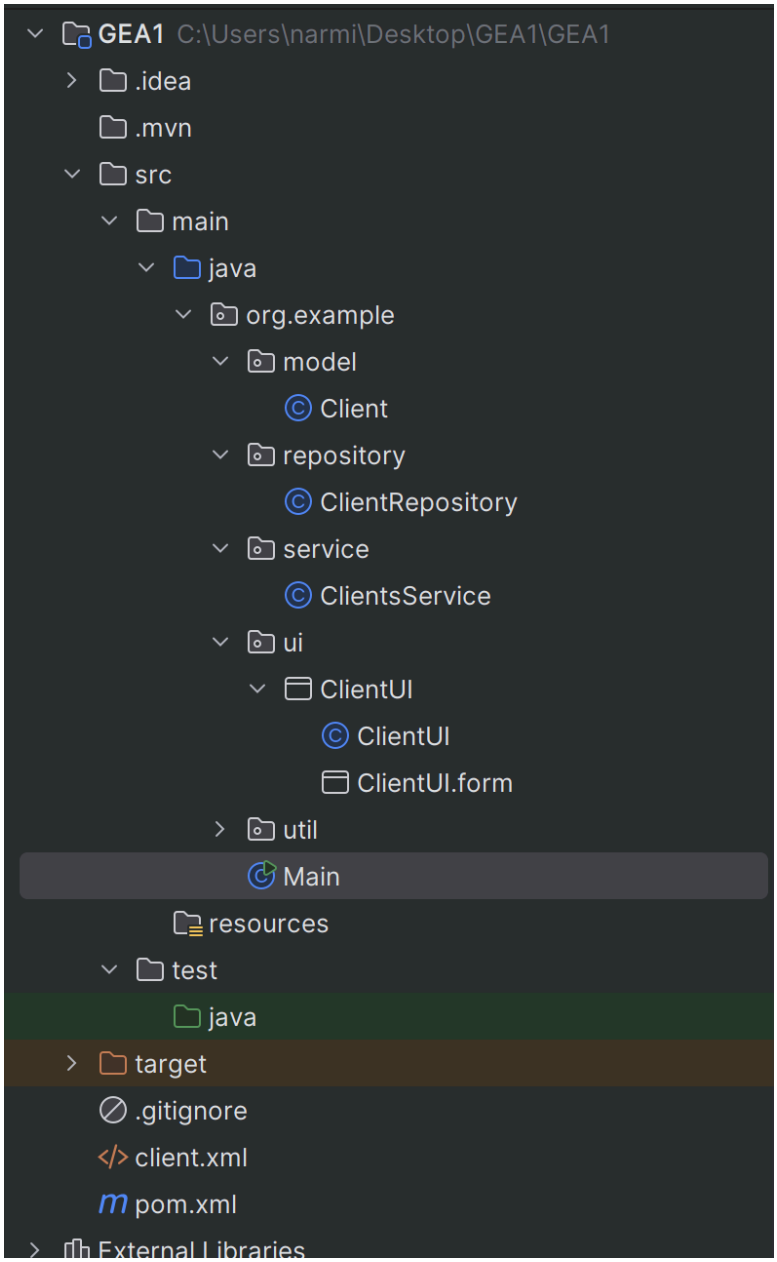
Ce projet a pour but de développer une **application Java de gestion des adhérents** pour la Maison des Ligues (M2L).
L'application permet d'enregistrer les membres, de calculer leurs cotisations et de sauvegarder toutes les informations dans des fichiers XML, sans utiliser de base de données.

L'architecture du projet suit le modèle **MVC simplifié** :

- **Model** : les classes de données (`Client`),
- **Repository** : la gestion de la persistance (`ClientRepository`, `XmlDataManager`),
- **Service** : la logique métier (`ClientsService`),
- **UI** : l'interface graphique (Swing).

Structure générale du projet

Package	Rôle principal
`org.example.model`	Définit les objets métiers (ex. : `Client`)
`org.example.repository`	Gère la lecture et l'écriture des fichiers XML
`org.example.service`	Contient la logique métier (calculs, gestion des adhésions)
`org.example.util`	Contient les outils de gestion du XML (`XmlDataManager`, `LocalDateAdapter`)
`org.example.ui`	Interface graphique Swing



Le projet utilise **Maven** avec un fichier `pom.xml` pour gérer les dépendances et structurer le code proprement.

Classe `Client` (model)

Les annotations **JAXB** sont utilisées pour relier la classe Java à la structure du fichier XML.

Les annotations `@XmlElement` permettent de lier directement les attributs de la classe `Client` aux balises XML correspondantes.
Cela simplifie la lecture et l'écriture des fichiers XML grâce à JAXB, qui transforme automatiquement les objets Java en XML (et inversement).
L'annotation `@XmlRootElement` définit la balise principale `<client>`, et `@XmlAccessorType(XmlAccessType.FIELD)` indique que JAXB doit accéder directement aux champs privés de la classe.

```java
package org.example.model;

import javax.xml.bind.annotation.*;
import java.time.LocalDate;

@XmlRootElement(name = "client")  29 usages
@XmlAccessorType(XmlAccessType.FIELD)
public class Client {
    /**
     * J'ai utilisé un fichier pom.xml parce que mon projet utilise Maven.
     * Maven permet de gérer automatiquement les bibliothèques externes (comme celles pour lire ou écrire des fichiers XML) et de structurer le projet propr
     * Au lieu d'ajouter manuellement des fichiers .jar, le pom.xml télécharge les dépendances nécessaires et configure le projet de manière standard
     * */

    @XmlElement  2 usages
    private int id; // Identifiant

    @XmlElement  4 usages
    private String nom; // Prénom

    @XmlElement  4 usages
    private String prenom; // Nom

    // Pour JAXB : stocké en String, converti en LocalDate via getter/setter
    @XmlElement  3 usages
    private String dateNaissance; // Date de naissance

    @XmlElement  2 usages
    private String lieuNaissance; // Lieu de naissance

    @XmlElement  4 usages
    private String sexe; // Sexe

    @XmlElement  2 usages
    private String nationalite; // Nationalité
```

Rôle :

- Représente un adhérent avec toutes ses informations (identité, coordonnées, assurance, cotisation, etc.).
- Fournit des méthodes pour déterminer la **catégorie d'âge** selon l'année de naissance.
- Gère le type `LocalDate` pour la date de naissance via un adaptateur JAXB.

Classe `ClientRepository` (repository)

La classe `ClientRepository` sert à gérer les accès aux données du fichier `client.xml`.
Elle utilise la classe `XmlDataManager` pour lire et écrire les données XML, ce qui sépare clairement la logique de stockage (repository) de la logique métier (service).
Ainsi, si un jour on remplace les fichiers XML par une base de données, on n'aura qu'à modifier cette classe sans toucher au reste du projet.

```java
package org.example.repository;


import org.example.model.Client;
import org.example.util.XmlDataManager;

import java.util.List;

/***
 * La classe ClientRepository sert à gérer les accès aux données du fichier client.xml.
 * Elle utilise la classe XmlDataManager pour lire et écrire les données XML, ce qui sépare clairement la logique de stockage (repository) de la logique métier (service).
 * Ainsi, si un jour on remplace les fichiers XML par une base de données, on n'aura qu'à modifier cette classe sans toucher au reste du projet.
 */

public class ClientRepository {  3 usages
    private static final String FILE_PATH = "client.xml";  5 usages

    public List<Client> findAll() { return XmlDataManager.clientsLu(FILE_PATH); }

    public void save(Client client) {  2 usages
        List<Client> clients = XmlDataManager.clientsLu(FILE_PATH);
        clients.add(client);
        XmlDataManager.clientsEcrit(FILE_PATH, clients);
    }


    public void delete(int id) {  2 usages
        List<Client> clients = XmlDataManager.clientsLu(FILE_PATH);
        clients.removeIf( Client m -> m.getId() == id);
        XmlDataManager.clientsEcrit(FILE_PATH, clients);
    }
}
```

Méthodes principales :

- `findAll()` : lit tous les clients du fichier XML.
- `save(Client client)` : ajoute un nouveau client et met à jour le fichier XML.
- `delete(int id)` : supprime un client par son identifiant.

Classe `ClientsService` (service)

Service de gestion des membres FFE (Fédération Française d'Escrime).
Ce service s'occupe :

- de l'inscription des membres du club d'escrime,

- du calcul des cotisations,
- et de la gestion des adhésions.

```java
import org.example.repository.ClientRepository;
import java.util.List;

/**
 * Service de gestion des membres FFE (Fédération Française d'Escrime)
 *
 * Ce service s'occupe :
 * - de l'inscription des membres du club d'escrime,
 * - du calcul des cotisations,
 * - et de la gestion des adhésions.
 *
 * Structure des tarifs :
 * - Inscription + Adhésion : varie selon la catégorie d'âge (190€, 220€, 255€)
 * - Licence FFE : varie selon la catégorie d'âge (28€, 45€, 55€)
 * - Options d'assurance : normale, sans assurance (-0,21€), assurance renforcée (+1,49€)
 * - Réductions famille : réduction différente pour le 2e et le 3e membre (et plus)
 * - Carte Tireur Expérimenté : 95€ (10 séances)
 */
public class ClientsService {  3 usages

    private ClientRepository repository = new ClientRepository();  8 usages

    /**
     * Enregistre un nouveau membre et calcule ses cotisations
     */
    public void registerClient(Client client) {  2 usages
        calculerEtAttribuerMontant(client);
        repository.save(client);
    }

    /**
     * Récupère la liste de tous les membres
     */
    public List<Client> getTousLesClients() {  1 usage
```

Structure des tarifs :

Élément	Description
Inscription + Adhésion	Varie selon la catégorie d'âge (190€, 220€, 255€)
Licence FFE	Varie selon la catégorie d'âge (28€, 45€, 55€)
Assurance	Normale, sans assurance (-0,21€) ou renforcée (+1,49€)
Réductions famille	Différente pour le 2e et le 3e membre
Carte Tireur Expérimenté	95€ (10 séances)

Rôle :

- Calculer le montant total à payer pour chaque adhérent.
- Gérer les réductions familiales et les options d'assurance.
- Produire un résumé détaillé des cotisations.

Classe `LocalDateAdapter` (utilitaire)

On a créé `LocalDateAdapter` pour apprendre à convertir automatiquement les dates (`LocalDate`) en texte quand on lit ou écrit des fichiers XML avec JAXB.
En effet, JAXB ne sait pas gérer directement le type `LocalDate`, donc cette classe agit comme un **traducteur** entre le format XML (texte) et le format Java (`LocalDate`).

```java
package org.example.util;

import javax.xml.bind.annotation.adapters.XmlAdapter;
import java.time.LocalDate;

/***
 * On a créé LocalDateAdapter pour apprendre à convertir automatiquement les dates (LocalDate) en texte quand on lit ou écrit des fichiers XML avec JAXB.
 */

public class LocalDateAdapter extends XmlAdapter<String, LocalDate> {  no usages
    @Override
    public LocalDate unmarshal(String v) { return LocalDate.parse(v); }
    @Override
    public String marshal(LocalDate v) { return v.toString(); }
}
```

Fonctionnement :

- `unmarshal(String v)` : transforme une date texte du XML (ex. `"2005-07-25"`) en `LocalDate`.
- `marshal(LocalDate v)` : convertit un objet `LocalDate` en texte pour l'écriture XML.

Classe `XmlDataManager` (utilitaire de gestion XML)

La classe `XmlDataManager` permet de gérer la persistance des données sous format XML.

Elle utilise la bibliothèque JAXB pour convertir automatiquement les objets `Client` en fichier XML et inversement.
Une classe interne `MembersWrapper` sert de conteneur pour la liste des clients, car JAXB nécessite un élément racine unique dans le fichier.
Cette approche rend la sauvegarde et la lecture des données simples et automatiques.

```java
public class XmlDataManager {  6 usages

    /**
     * La classe XmlDataManager permet de gérer la persistance des données sous format XML.
     * Elle utilise la bibliothèque JAXB pour convertir automatiquement les objets Client en fichier XML et inversement.
     * Une classe interne MembersWrapper sert de conteneur pour la liste des clients, car JAXB nécessite un élément racine unique dans le fichier.
     * Cette approche rend la sauvegarde et la lecture des données simples et automatiques.
     * */

    public static List<Client> clientsLu(String chemin) {  3 usages
        try {
            File file = new File(chemin);
            if (!file.exists()) return new ArrayList<>();

            JAXBContext context = JAXBContext.newInstance(MembersWrapper.class);
            Unmarshaller unmarshaller = context.createUnmarshaller();
            MembersWrapper wrapper = (MembersWrapper) unmarshaller.unmarshal(file);


            if (wrapper.getClients() == null) {
                return new ArrayList<>();
            }

            return wrapper.getClients();
        } catch (Exception e) {
            e.printStackTrace();
            return new ArrayList<>();
        }
    }

    public static void clientsEcrit(String chemin, List<Client> clients) {  2 usages
        try {
            JAXBContext context = JAXBContext.newInstance(MembersWrapper.class);
            Marshaller marshaller = context.createMarshaller();
            marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
```

Méthodes principales :

- `clientsLu(String chemin)` → lit le fichier XML et renvoie une liste d'objets `Client`.

- `clientsEcrit(String chemin, List<Client> clients)` → écrit la liste de clients dans le fichier XML.

Pourquoi Maven (pom.xml)

J'ai utilisé un fichier `pom.xml` parce que mon projet utilise **Maven**.
Maven permet de gérer automatiquement les bibliothèques externes (comme celles pour lire ou écrire des fichiers XML) et de structurer le projet proprement.
Au lieu d'ajouter manuellement des fichiers `.jar`, le `pom.xml` télécharge les dépendances nécessaires et configure le projet de manière standard.

On voulait que notre projet soit bien structuré, donc on a cherché une solution moderne pour gérer les bibliothèques Java. On a découvert Maven et on a appris à l'utiliser grâce à des tutoriels — ce n'était pas au programme, mais ça nous a beaucoup aidés.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>GEA1</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
    <dependencies>
        <dependency>
            <groupId>javax.xml.bind</groupId>
            <artifactId>jaxb-api</artifactId>
            <version>2.3.1</version>
        </dependency>

        <dependency>
            <groupId>org.glassfish.jaxb</groupId>
            <artifactId>jaxb-runtime</artifactId>
            <version>2.3.1</version>
        </dependency>
    </dependencies>

</project>
```

Avantages :

- Gestion simplifiée des dépendances.
- Structure de projet standardisée.

- Compilation et exécution plus faciles.

Class Main:

```java
public class Main {
    public static void main(String[] args) {
        ClientsService service = new ClientsService();

        // ===== 1. Création d'un membre =====
        Client client1 = new Client();
        client1.setId(1);
        client1.setNom("Jean");
        client1.setPrenom("Dupont");
        client1.setDateNaissance(LocalDate.of( year: 2010,  month: 5,  dayOfMonth: 14)); // Né en 2010
        client1.setSexe("Homme");
        client1.setTypeArme("Épée");
        client1.setAssure(true); // Assuré
        client1.setAssurenceRenforcee(false); // Assurance renforcée désactivée
        client1.setNbrClientFamille(1); // Nombre de membres dans la famille
        client1.setCarteTiteurExprimante(0); // Carte d'essai expérimenté

        // Enregistrer le membre et calculer les frais
        service.registerClient(client1);

        // Afficher le résumé des frais du membre
        System.out.println(service.genererRecapitulatif(client1));

        // ===== 2. Exemple d'un membre de la famille =====
        Client client2 = new Client();
        client2.setId(2);
        client2.setNom("Marie");
        client2.setPrenom("Dupont");
        client2.setDateNaissance(LocalDate.of( year: 2012,  month: 8,  dayOfMonth: 2));
        client2.setSexe("Femme");
        client2.setTypeArme("Fleuret");
        client2.setAssure(false);            // Non assuré
        client2.setAssurenceRenforcee(false);  // Assurance renforcée désactivée
        client2.setNbrClientFamille(2);      // Deuxième membre de la famille
        client2.setCarteTiteurExprimante(95);  // Carte d'essai expérimenté
```

Console:
1. Client

```
  Jean Dupont - RÉCAPITULATIF
═══════════════════════════════════════

Date de naissance : 2010-05-14
Catégorie d'âge : E13-E20 (2003-2011)
Sexe : Homme

DÉTAILS DES TARIFS :
───────────────────────────────────────
Inscription + Adhésion :   220,00 €
Licence FFE :               55,00 €

───────────────────────────────────────
MONTANT TOTAL :            275,00 €
═══════════════════════════════════════
```

2. Client (La famille)

```
═══════════════════════════════════════
  Marie Dupont - RÉCAPITULATIF
═══════════════════════════════════════

Date de naissance : 2012-08-02
Catégorie d'âge : M11 (2012-2013)
Sexe : Femme

DÉTAILS DES TARIFS :
───────────────────────────────────────
Inscription + Adhésion :   190,00 €
Licence FFE :               44,79 €
  (Sans assurance : -0,21 €)

Réduction famille (2ème membre) :  -24,00 €

Carte Tireur Expérimenté :   95,00 €
  (Inscription + adhésion 10 séances)

───────────────────────────────────────
MONTANT TOTAL :            305,79 €
═══════════════════════════════════════
```

Tous les membres:

```
Tous les membres :
Marie Dupont (M11 (2012-2013)) - Femme - Total: 305,79 €
Jean Dupont (E13-E20 (2003-2011)) - Homme - Total: 275,00 €
```

Conclusion

Ce projet nous a permis de comprendre :

- comment structurer une application Java en couches (modèle, service, repository),

- comment utiliser **JAXB** pour gérer des fichiers XML,
- et comment utiliser **Maven** pour automatiser la gestion des dépendances.

C’est un projet complet et formateur, qui montre le fonctionnement d’une application de gestion réelle.